

Arithmetic Without Numbers

What happens inside an LLM when it tries to calculate with nothing but matrices

ALVARO VIDELA

Rune studies whether arithmetic structure inside a language model can be read and used without parsing the prompt text. The strongest current result is an activation-derived tool-argument route on Llama-3.1-8B: operation and operand readouts are decoded from captured activations, then Python computes only after the decoded tuple exists. This is not a claim of native arithmetic repair or residual-state JIT replacement. It is a narrower result about internal representations, provenance, and the boundary between reading a mechanism and using it safely.

1 THE QUESTION

If you learned arithmetic the ordinary human way, you probably learned it with a body. You counted on fingers, grouped things into piles, lined digits into columns, and carried a one. Human arithmetic is full of objects: fingers, beads, marks, columns, strokes, and places.

A language model has none of that. It has matrices.

At each layer, a grid of numbers transforms another grid of numbers. Tokens enter, activations flow, logits come out. No fingers. No abacus. No column of digits written on a page. And yet, if you ask a modern language model for the greatest common divisor of two numbers, or a multiplication, or a division with remainder, something inside that matrix-only body responds.

Sometimes it answers correctly. Often it does not. The question that started Rune was simple to ask and hard to answer: how does a language model do arithmetic, if all it has are vectors?

The first temptation is to treat this as a product problem. If the model is bad at arithmetic, call a calculator. That works. It is also not very mysterious. A parser can read “what is 84 times 37,” translate it into $84 * 37$, send that expression to Python, and return the result. That general family of external-computation routes is well established: PAL and Program of Thoughts ask models to produce executable programs, ReAct interleaves reasoning with actions, and Toolformer studies models learning when and how to call APIs [2, 4, 9, 11].

Rune was chasing a different question. Could we look inside the model and find the calculation it was trying to perform? Could the model’s own internal states tell us the operation and operands? And, if so, could we use that information without cheating by reading the prompt text directly?

2 A DIFFERENT KIND OF TOOL USE

The original dream was more ambitious: a mechanism-aware just-in-time compiler for model arithmetic. The ideal pipeline looked like this. The model reads an arithmetic prompt; we identify the internal mechanism it is using; we replace the unreliable part with exact computation; then the model continues naturally, as if it had done the math itself.

That is a beautiful idea. It is also not what Rune proves.

What survived the experiments was narrower, but more interesting than ordinary tool use. In a frozen Llama model, internal activations expose enough structure to recover the arithmetic operation and operands under a strict no-parser rule. The runtime is not allowed to use regexes, prompt text, hidden labels, command-line operands, or gold answers. It gets token IDs and activations. From those activations, it must infer: this is gcd, or lcm, or multiplication, or division with remainder; these are the two numbers. Only after that decoded tuple exists may Python compute.

The calculator is not the surprising part. The surprising part is that the arguments to the calculator can come from inside the model.

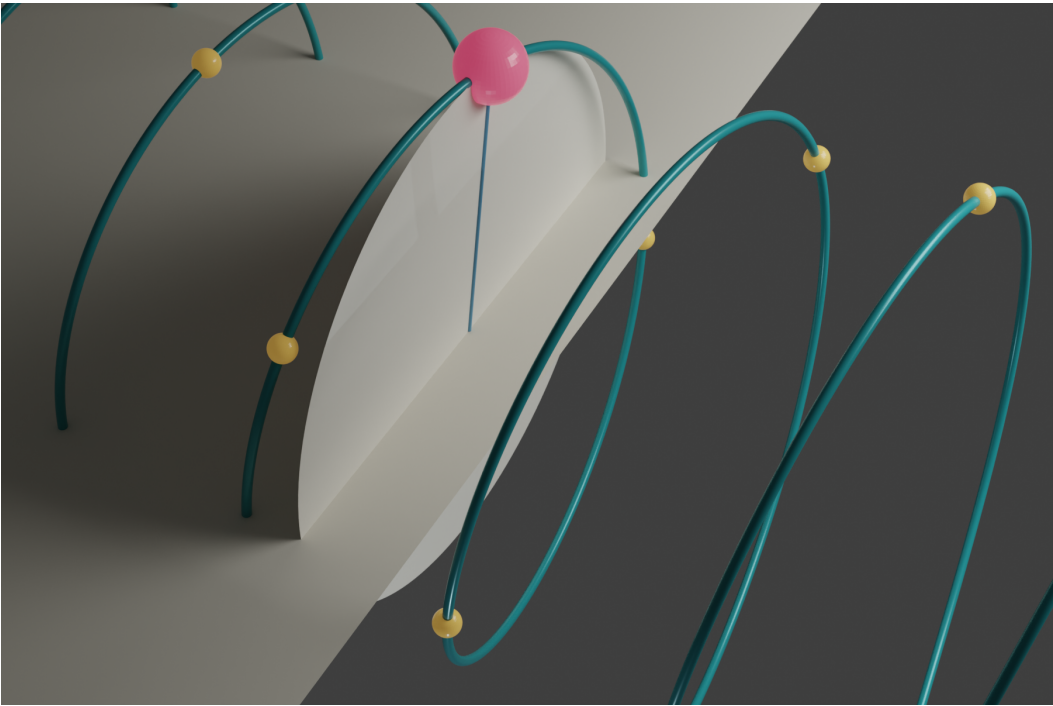


Fig. 1. A visual analogy for integer phase geometry in a matrix-only model: value can be represented by position on rotating directions rather than by fingers, beads, or written columns.

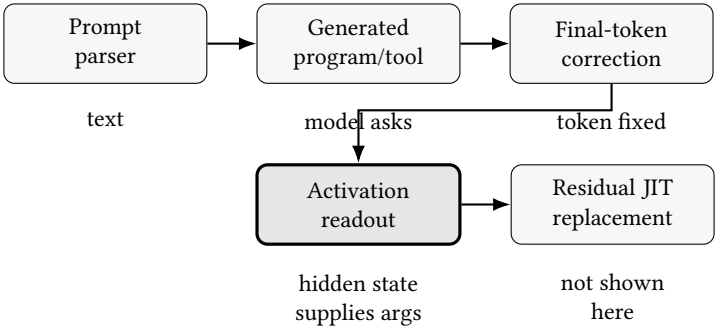


Fig. 2. Claim ladder for the same visible answer. Rune’s supported claim is the highlighted activation-readout route: the calculator arguments come from hidden vectors, not from prompt parsing or an already-known answer.

That distinction matters because a normal tool route can solve the problem without learning anything about model internals. Rune asks whether the model’s residual stream already carries the calculation description, even when the model’s own next-token answer would be wrong.

3 THE MATRIX BODY

George Lakoff and Rafael Núñez argued that human mathematics is built from embodied experience: grouping, moving, measuring, collecting, and balancing [6]. A transformer has a different body. Its body is matrices, residual streams, attention maps, and learned projections. If human arithmetic can grow out of fingers and spatial metaphors, what kind of arithmetic grows out of a matrix-only body?

Recent mechanistic work suggests a geometric answer. Kantamneni and Tegmark argued that language models can represent integers on generalized helices and use trigonometric structure for addition [5]. Stolfo, Belinkov, and Sachan used causal mediation analysis to study arithmetic mechanisms in transformers [10]. Other work, including Nikankin and colleagues’ “bag of heuristics” account, warns that model arithmetic is often not a clean schoolbook algorithm but a mixture of learned features and prompt-sensitive heuristics [7].

Rune builds on that line rather than originating it. The contribution here is a scoped reproduction-and-extension story around activation-derived tool arguments, provenance controls, and resolution limits. We found Fourier-like readouts and value-like states, but also brittleness, prompt sensitivity, and failures that look less like a perfect algorithm than a finite-resolution internal geometry.

3.1 A Vector Changes as the Model Reads

Imagine reading `What is the gcd of 84 and 36?` one token at a time. The model does not create neat variables named `operand_a` and `operand_b`. Each token position carries a long vector. As the prompt passes through transformer layers, those vectors are updated repeatedly. Some updates move information across positions: the token for 36 can affect the state near the answer position. Other updates reshape the local state: a direction in the vector may become more gcd-like, more operand-like, or more answer-like.

The terms are less exotic in context. A token is one unit the model reads or prints; for numbers it may be a chunk of digits rather than a full integer. The activation is the temporary vector at a token position. The residual stream is the running vector passed from layer to layer, a shared scratchpad without named variables. Attention lets one position gather information from others; the MLP, or feed-forward block, then reshapes that position’s vector locally. At the end, logits score possible next tokens.

This is why readouts and patches are possible at all. If operation and operands leave traces in the residual stream, a small readout may recover them. If a state really matters, a patch may change behavior. If a state is writable, an intervention may guide the model. But those are increasingly strong claims, and the vector itself does not label which claim is true.

4 NEXT-TOKEN PRESSURE

Next-token prediction changes the arithmetic problem. A human doing subtraction on paper usually works from the least significant digit toward the left: units first, then tens, then hundreds, carrying or borrowing as needed. A language model must emit text from left to right. For the answer 15696, it must commit to the visible prefix before the suffix exists. In tokenized form, the answer may come out as chunks like 15 then 696. The model is not filling in a scratch sheet from right to left; it is walking forward through the string.

That left-to-right constraint creates a rendering problem separate from the arithmetic problem. The model may have partial information about the answer but still lose precision as it emits longer strings. In Rune’s subtraction scaling experiment, exact greedy generation stayed at 96.7 percent for 6-digit subtraction, fell to 63.3 percent at 10 digits, reached 53.3 percent at 13 digits, and crossed below 50 percent at 14 digits. At 24 digits it was down to 6.7 percent.

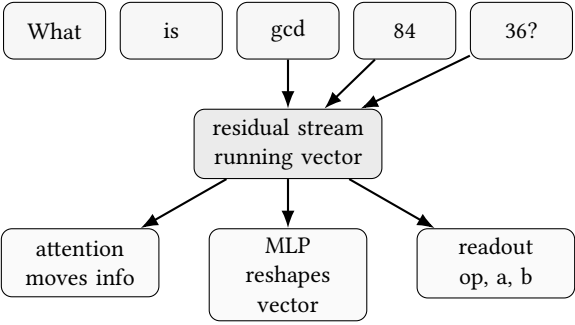


Fig. 3. The residual stream is a running vector, not a named variable table. Attention can move operand information across positions; the MLP reshapes the local vector; a readout asks whether facts such as gcd, 84, 36 are recoverable from that state.

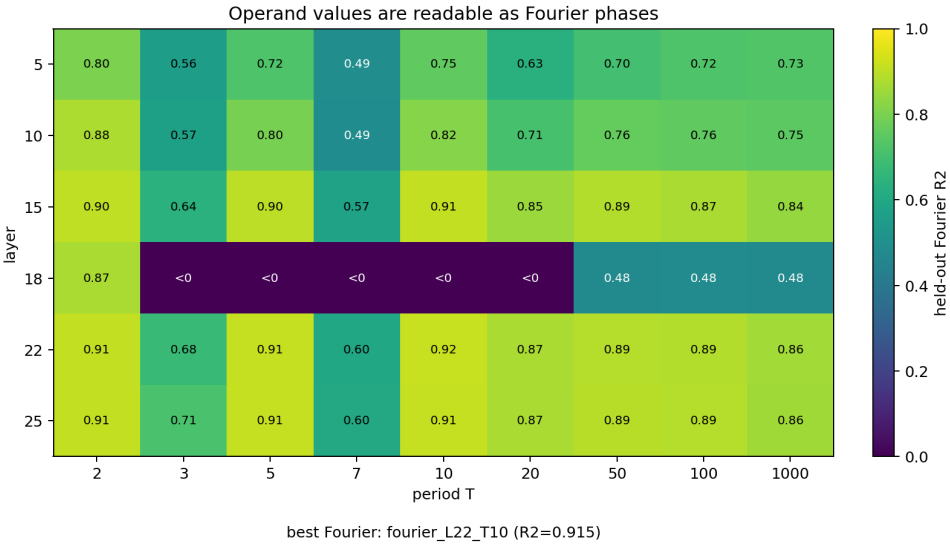


Fig. 4. Operand readouts are one way to ask whether number-like information is present in activations. A readout can show information is recoverable; it does not by itself prove the model used that information to compute.

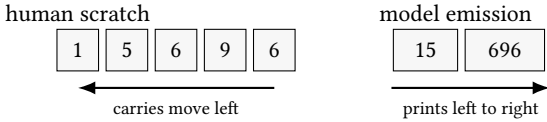


Fig. 5. Humans can compute from the rightmost digit and write the answer later. A next-token model must emit the visible prefix first. That makes numeric rendering a separate pressure from internal arithmetic.

A separate counting experiment made the same pressure visible in a simpler setting. The model saw runs of large consecutive numbers and had to print the next one. An easy case simply advances

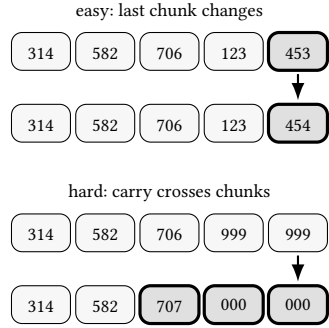


Fig. 6. The counting experiment exposed the same left-to-right pressure in token chunks. The easy case edits one final chunk; the hard case must update several chunks before the model has printed them.

Table 1. Counting continuation example. The model tokenizer splits the visible number into chunks; the hard case requires a carry across two chunk boundaries.

Case	Chunks									
Easy next	314	582	706	123	453	->	314	582	706	123 454
Deep carry	314	582	706	999	999	->	314	582	707	000 000
Stuck-copy error	314	582	706	999	999	->	314	582	706	999 999

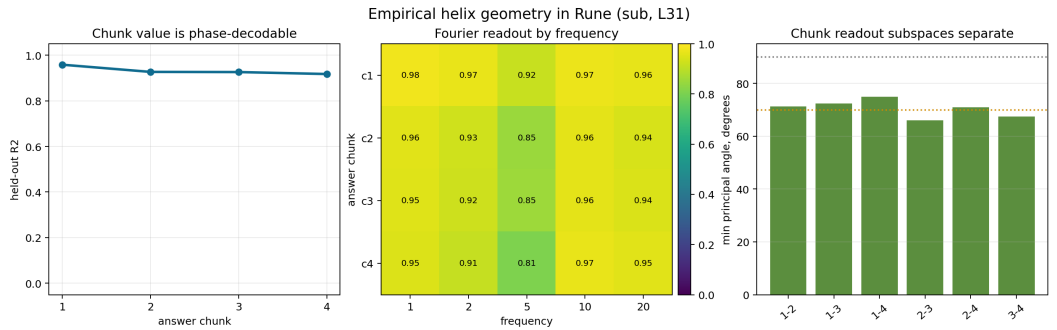


Fig. 7. Empirical helix-resolution evidence for multi-chunk subtraction. Longer answers remain partly readable, but adjacent chunk subspaces become more crowded and less forgiving.

the last chunk. A deep-carry case must change multiple chunks at once. Cases like this collapsed; the best deep-carry cell reached only 18.75 percent accuracy, and the dominant error was stuck-copy.

The helix-resolution experiments gave a more mechanistic picture of that boundary. For 12-digit subtraction answers split into four 3-digit chunks, each chunk remained strongly phase-decodable at layer 31: R^2 was about 0.96 for chunk 1 and about 0.92 for chunk 4. So the signal did not simply vanish. The subtler finding was crowding. Adjacent chunk readout subspaces were not cleanly orthogonal; the minimum principal angle dropped as low as 66 degrees. In a 14-digit, five-chunk follow-up, chunks 2–4 lost R^2 and adjacent angles tightened further, for example c3–c4 went from 67.4 degrees to 58.8 degrees. Two of three preregistered crowding predictions passed.

Table 2. Subtraction exact-match rate by answer digit count. Source: `cd_e10_operand_scaling`, 30 greedy generations per digit band.

Digits	Exact	Prefix correct	Mean edit distance
6	96.67%	97.2%	0.10
10	63.33%	80.7%	0.60
13	53.33%	73.8%	1.07
14	43.33%	67.1%	1.07
16	33.33%	64.6%	1.40
24	6.67%	17.6%	8.77

Table 3. Adjacent chunk subspace angles tightened in the 14-digit follow-up. Smaller angles mean less separation between nearby readout directions.

Chunk pair	12-digit	14-digit
c1–c2	71.3°	65.3°
c2–c3	66.0°	60.2°
c3–c4	67.4°	58.8°

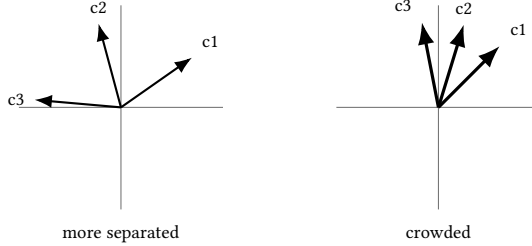


Fig. 8. Crowding is a geometric resolution problem. In the experiments this was measured with principal angles between chunk readout subspaces; the drawing is only a visual guide to what smaller angles mean.

So “the helix gets saturated” is too crude, but “there is a resolution budget” is fair. Longer answers still live in readable geometry, but the geometry becomes more crowded and less forgiving.

5 WHAT FAILED USEFULLY

At first, this looked like a path toward the compiler dream. If we could write the right answer state into the model, perhaps the model would continue from there.

The evidence sharpened that hope into something more modest. Making a model emit a known answer is not the same as making the model compute. If the experiment already knows the answer, and then uses that answer to choose a steering vector, it has measured the model’s ability to render a supplied value. That is useful, but it is not arithmetic understanding, and it is not an honest deployment pipeline.

This was one of the first major lessons: token rendering can masquerade as computation.

The residual-write story also became clearer. We tried to write corrected answer information into the model’s residual stream and let it continue. For the tested single-site writes, residual interventions had no accuracy advantage over simpler token or logit correction. Worse, they disturbed

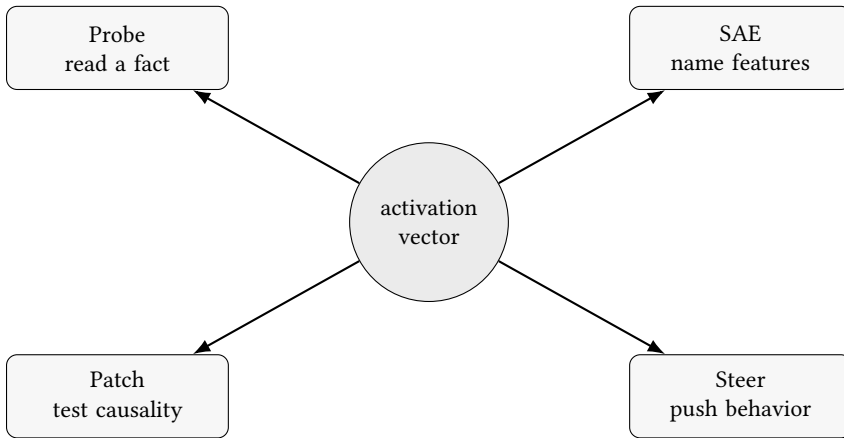


Fig. 9. The same activation can be inspected in different ways. The tools do not make the same claim: readout, feature naming, causal testing, and steering are different standards of evidence.

surrounding behavior more. That did not prove residual replacement is impossible. It proved something more practical: a readable variable is not necessarily a writable register.

Mechanistic interpretability often celebrates reading: decode a concept, localize a feature, find a direction. Sparse autoencoders and dictionary learning are one important approach to naming such features in dense activation spaces [1, 3]. Activation patching is one common way to move from readout toward causal evidence, but patching methodology has its own pitfalls and metric choices [12]. Engineering wants an even stronger property: change the state, preserve behavior, continue execution. Those are different problems.

6 THE INSPECTION TOOLBOX

Most of Rune was not one big experiment. It was a toolbox applied repeatedly, with stricter controls each time. The tools are useful, but each answers a different question.

This is why the article keeps distinguishing readout, causality, and writable replacement. A probe asks, “Can I read something?” An SAE asks, “Can I name the parts?” A patch asks, “Does this part matter?” Steering asks, “What happens if I push it?” Confusing those questions is how interpretability experiments accidentally overclaim.

7 THE ROUTE THAT SURVIVED

The final system used activation-only gates and operand readouts. It had to decide when an arithmetic route was safe to fire, identify the operation, identify the operand pair, and then call Python only after the decoded tuple was formed. The result was evaluated on frozen benchmark slices, with thresholds locked and provenance tracked.

On a broad benchmark with the Llama weights kept frozen, the route passed across four operations: multiplication, division with remainder, gcd, and lcm. Here “passed” means two things at once. On real arithmetic prompts, the route had to fire: a gate had to decide that the calculator was allowed to run, and the operation and operands had to come from activations. On adversarial prompts, written to tempt the route into doing the wrong thing, it had to stay silent. Across 11,736 locked examples—with examples, thresholds, and scoring rules fixed before the final aggregate—and 1,536 targets, it produced large exact-answer lifts with zero fires on the constructed hard-negative suite used in this audit. A hard negative is a deliberately tricky no-fire prompt: it may

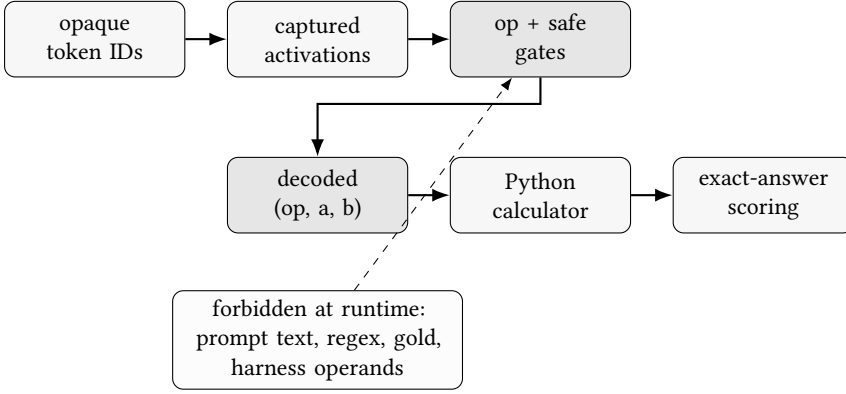


Fig. 10. The surviving route is activation-derived tool use. The calculator is ordinary; the provenance question is whether its arguments came from hidden state rather than forbidden prompt-derived fields.

Table 4. Broad frozen arithmetic/adversarial cross-seed aggregate on Llama-3.1-8B. Lift is exact-answer improvement over the model’s native answer on target prompts. False-fire is the rate of unwanted calculator calls on prompts where the route should stay silent.

Operation	Routed	Lift	False-fire rate
multiplication	0.865	0.852	0.000
division with remainder	0.909	0.693	0.000
lcm	0.969	0.966	0.000
gcd	0.922	0.594	0.000

Table 5. Recognized DeepMind source slice. The route calculated many more exact answers than the frozen model produced by itself.

Operation	Routed exact	Exact-answer lift
division with remainder	0.992	+0.810
gcd	1.000	+0.502
lcm	0.980	+0.968

contain quoted arithmetic, code, a table of numbers, or an instruction saying not to compute, but the correct behavior is no calculator call (Table 4).

On a recognized source slice from the DeepMind Mathematics Dataset of Saxton and colleagues [8], the result covered three operations: gcd, division with remainder, and lcm. Recognized means the audit could map a dataset prompt to one of the supported forms: two integer operands, a supported operation, and a checkable answer. It is a coverage filter, not a claim that every DeepMind prompt was understood or supported. Multiplication was not claimed there because the source filtering did not produce enough accepted two-integer multiplication examples for a powered result. Across 3,822 locked examples and 1,233 targets, the activation-derived route again produced strong exact-answer lifts with zero recorded false fires.

The safety and provenance work mattered as much as the benchmark lifts. The word provenance is doing real work here: it is the audit trail for where the calculator arguments came from. We ran

Table 6. Examples of the fire/no-fire boundary. The route should fire on arithmetic requests, but not merely because arithmetic-looking text appears.

Should fire	Should not fire
Calculate the highest common factor of 5924 and 1024.	She wrote 'gcd(48, 18) = 6' on the whiteboard and then changed the subject to budgets of 200 and 300.
What is the remainder when 7696 is divided by 5130?	A reporter typed '144 / 12' into her notes but the story was about a basketball game.
What is the smallest common multiple of 4740 and 1152?	The chart showed 6, 12, 18, 24 as factor labels but the article discussed musical notation.

Table 7. Independent hard-negative audit. These are prompts where the desired behavior is no calculator call. Zero fires is therefore a good result inside this scoped audit.

Category	Fires / examples
decimals, signs, out of domain	0 / 1200
distractor-heavy number text	0 / 600
do-not-compute prompts	0 / 1200
quoted arithmetic	0 / 1200
tables, logs, code, invoices	0 / 2400
wrong operation, same numbers	0 / 3600

a full replay audit over 15,558 runtime bundles, excluding forbidden fields such as prompt text, regex outputs, decoded token spans, harness operands, operation labels, and gold answers. The route had to reproduce from allowed runtime artifacts. It passed with zero replay failures.

We also ran an independent hard-negative audit, using no-fire cases generated separately from the positive arithmetic benchmark. The categories included quoted arithmetic, do-not-compute prompts, wrong-operation prompts with the same numbers, tables, logs, code, invoices, distractor-heavy number text, decimals, signs, and out-of-domain cases. Across 10,200 non-trigger examples, the route did not fire.

8 THE BOUNDARY

There is causal support, but it has to be said carefully. In causal interchange tests, selected internal operand chunks could be patched from donor examples into recipient examples, and the decoded operands and routed calculator answers followed the donor. This uses the same broad causal-intervention discipline as activation patching and causal mediation work, not a new causal methodology invented here [10, 12]. It supports the claim that those internal chunks are causally involved in the decoded tuple. In the final DeepMind causal summary, division with remainder cleared the powered pair-count gate. Gcd and lcm had perfect rates in the available pairs, but too few pairs to clear the frozen causal gate. So the causal evidence is supportive, not fully powered across every final operation.

The cross-model story is also humbling. A real Qwen operand-localization sweep failed. The current Llama route did not transfer. That is not a footnote. It is a lesson: internal activation routes

are not portable like text parsers. A parser sees the same string. A model’s internal geometry may be entirely different.

The next steps are concrete: build model-specific operand localizers instead of assuming transfer; complete causal interchange tests where source coverage allows it; compare any residual-write attempt against boring logit and parser baselines; and keep the no-parser replay boundary as a non-negotiable test.

The final claim is deliberately modest: a frozen Llama model’s activations can supply operation and operand arguments for an exact calculator under an opaque, no-parser runtime on the current benchmark slices.

That is not native arithmetic repair. It is not a general cross-model result. It is not behavior-preserving residual JIT compilation. But it is a useful boundary. It shows that the model’s hidden state can contain the calculation even when its output would be wrong. It shows that mechanistic interpretability can contribute to tool systems, if the provenance rules are strict enough. And it shows how easy it is to overclaim unless the experiment is designed to catch you.

9 WHAT TO TAKE AWAY

Models really do build arithmetic-shaped internal structure. Not arithmetic as humans experience it, with fingers and written columns, but arithmetic as a geometry of directions, rotations, periodic features, and value-like residual states. A matrix-only body can invent ways to represent divisibility, magnitude, chunks, and numeric emission.

Reading those structures is easier than turning them into a robust system. A probe can look good for the wrong reason. A steering vector can render an answer without proving computation. A residual write can change a token while damaging the model’s state. The interesting science is not the first positive result; it is the sequence of controls that survives.

Tool use has layers. The easy layer is text-driven: parse the prompt, call the tool. The harder layer is activation-derived: prove that the tool arguments came from the model’s internal state. The hardest layer is internal replacement: write the result back into the model and preserve behavior. Rune does not solve the hardest layer. It makes progress on the middle one.

If you want to build systems from those traces, do not trust the first beautiful signal. Ask what else it could be measuring. Remove the prompt text. Remove the labels. Replay without the forbidden fields. Add the annoying negatives. Compare against the boring baseline. Separate what the model represents from what you can safely write.

That is how the story gets smaller. It is also how it becomes true.

REFERENCES

- [1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. 2023. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. *Transformer Circuits Thread* (2023). <https://transformer-circuits.pub/2023/monosemantic-features/>
- [2] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023). <https://doi.org/10.48550/arXiv.2211.12588> arXiv:2211.12588.
- [3] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. 2023. Sparse Autoencoders Find Highly Interpretable Features in Language Models. *arXiv preprint arXiv:2309.08600* (2023). <https://doi.org/10.48550/arXiv.2309.08600>
- [4] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2022. PAL: Program-aided Language Models. *arXiv preprint arXiv:2211.10435* (2022). <https://doi.org/10.48550/arXiv.2211.10435>

- [5] Subhash Kantamneni and Max Tegmark. 2025. Language Models Use Trigonometry to Do Addition. *arXiv preprint arXiv:2502.00873* (2025). <https://doi.org/10.48550/arXiv.2502.00873>
- [6] George Lakoff and Rafael E. Núñez. 2000. *Where Mathematics Comes From: How the Embodied Mind Brings Mathematics into Being*. Basic Books, New York, NY.
- [7] Yaniv Nikankin, Anja Reusch, Aaron Mueller, and Yonatan Belinkov. 2024. Arithmetic Without Algorithms: Language Models Solve Math With a Bag of Heuristics. *arXiv preprint arXiv:2410.21272* (2024). <https://doi.org/10.48550/arXiv.2410.21272>
- [8] David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. 2019. Analysing Mathematical Reasoning Abilities of Neural Models. *International Conference on Learning Representations* (2019). <https://doi.org/10.48550/arXiv.1904.01557> arXiv:1904.01557.
- [9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761* (2023). <https://doi.org/10.48550/arXiv.2302.04761>
- [10] Alessandro Stolfo, Yonatan Belinkov, and Mrinmaya Sachan. 2023. A Mechanistic Interpretation of Arithmetic Reasoning in Language Models using Causal Mediation Analysis. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (2023). <https://doi.org/10.48550/arXiv.2305.15054> arXiv:2305.15054.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations* (2023). <https://doi.org/10.48550/arXiv.2210.03629> arXiv:2210.03629.
- [12] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. *International Conference on Learning Representations* (2024). <https://doi.org/10.48550/arXiv.2309.16042> arXiv:2309.16042.